

Indian Wildlife Identifier: Zero-Shot Classification using Vision-Language Models

SMAI Assignment 3 – T7.5 Report

May 2, 2026

Abstract

Wildlife monitoring is a critical component of conservation efforts. Traditional methods relying on manual classification of camera-trap imagery are labor-intensive and not scalable. In this report, we present an automated Indian Wildlife Identifier system developed as part of SMAI Assignment 3 (T7.5). The system leverages the Contrastive Language-Image Pre-training (CLIP) model using a ViT-B/32 architecture to perform zero-shot classification across six Indian wildlife species: Cheetah, Fox, Hyena, Lion, Tiger, and Wolf. By utilizing prompt engineering, the zero-shot pipeline achieved an exceptional 98.03% Top-1 accuracy and 99.83% Top-3 accuracy on the provided dataset of over 1,700 images, all without any fine-tuning. Furthermore, we developed a production-ready, dark-themed web application using Streamlit that enriches model predictions with pre-cached Wikipedia summaries and IUCN conservation statuses, supporting both single-image analysis and batch camera-trap processing.

1 Introduction

The rapid decline of global biodiversity has necessitated the development of advanced technological solutions for wildlife tracking and conservation. Non-invasive monitoring tools, such as motion-activated camera traps, generate vast amounts of image data. Analyzing this data manually is an insurmountable bottleneck for ecologists and conservationists. Consequently, the integration of Artificial Intelligence (AI), specifically Computer Vision (CV), has become a priority in ecological research.

This project focuses on building an Indian Wildlife Identifier capable of accurately distinguishing between six prominent species: the Cheetah, Fox, Hyena, Lion, Tiger, and Wolf. While traditional deep learning approaches require training Convolutional Neural Networks (CNNs) from scratch or fine-tuning existing architectures (e.g., ResNet, EfficientNet) on domain-specific datasets, such methods demand substantial computational resources and time.

To address these limitations, we adopted a zero-shot learning paradigm utilizing OpenAI's CLIP (Contrastive Language-Image Pre-training) model. CLIP bridges the gap

between vision and natural language, allowing the model to classify images based on descriptive text prompts without seeing a single training example from our specific dataset.

The objectives of this project were threefold:

1. Evaluate the zero-shot capabilities of the `clip-vit-base-patch32` model on a curated wildlife dataset.
2. Implement advanced prompt engineering to maximize zero-shot accuracy.
3. Develop an accessible, user-friendly web interface that not only predicts the species but educates the user by providing Wikipedia summaries, fun facts, and color-coded International Union for Conservation of Nature (IUCN) red list statuses.

2 Dataset and Preprocessing

The dataset provided for this assignment consists of images categorized into six distinct wildlife classes: *Cheetah*, *Fox*, *Hyena*, *Lion*, *Tiger*, and *Wolf*.

2.1 Data Distribution

The dataset comprises a total of 1,725 images. The class distribution is generally balanced, ensuring that the evaluation metrics are not heavily skewed towards a majority class. The images vary significantly in their environmental context, lighting conditions, and the animal's pose, simulating real-world camera trap variance.

The dataset was organized into specific resolution subdirectories (e.g., 224x224, 300x300, 512x512). For the purpose of evaluating the CLIP model, all images across the resolution variants were aggregated, as the CLIP processor automatically handles dynamic resizing and center-cropping to match the model's expected input tensor size.

2.2 Data Integrity and Error Handling

During the initial data loading phase, it was discovered that a small number of files within the dataset were corrupted (e.g., 0-byte files or truncated image headers). Feeding these files directly into the PyTorch data pipeline would result in fatal exceptions.

To ensure robustness, a custom preprocessing script was implemented. During the directory traversal using the `pathlib` library, every image was first subjected to a verification check using the Python Imaging Library (`PIL.Image.verify()`). If an `OSError` or `UnidentifiedImageError` was caught, the file was flagged as corrupted, skipped, and logged to the console. This integrity check successfully filtered out 2 corrupted files, allowing the evaluation script to process the remaining 1,723 valid images uninterrupted.

3 Methodology

The core methodology revolves around deploying a foundational vision-language model in a purely zero-shot setting, circumventing the need for computationally expensive gradient updates.

3.1 Model Architecture: CLIP ViT-B/32

We utilized the `openai/clip-vit-base-patch32` model from the Hugging Face `transformers` library. CLIP consists of two separate neural network encoders:

- **Image Encoder:** A Vision Transformer (ViT) that divides the input image into 32×32 patches and projects them into a dense vector space.
- **Text Encoder:** A Transformer that encodes natural language descriptions into the same dense vector space.

During pre-training, CLIP learns to maximize the cosine similarity between the embeddings of an image and its corresponding text description within a massive batch, while minimizing the similarity with incorrect descriptions (contrastive loss).

3.2 Zero-Shot Inference Mechanism

In a zero-shot classification scenario, we define a set of potential text descriptions ($T_1, T_2, \dots, T_k$) corresponding to our $k = 6$ classes. When an unknown image I is uploaded: 1. The Image Encoder generates the embedding E_I . 2. The Text Encoder generates embeddings for all text prompts ($E_{T_1}, \dots, E_{T_k}$). 3. The cosine similarity is calculated between E_I and every text embedding. 4. A softmax function is applied over the similarity scores to produce a probability distribution. The class with the highest probability is selected as the prediction.

3.3 Prompt Engineering

A naive approach to zero-shot learning is to use single-word labels as prompts (e.g., “cheetah”). However, this often leads to ambiguity. For instance, the word “fox” could refer to the animal, a logo, or a person’s name.

To mitigate this, we employed “Prompt Engineering” to provide the text encoder with richer semantic context. The prompts were carefully crafted to include distinguishing physical features and geographic contexts:

- *Cheetah*: “a photo of a cheetah, a large spotted African cat”
- *Fox*: “a photo of a fox, a small reddish canine with a bushy tail”
- *Hyena*: “a photo of a hyena, an African carnivore with rounded ears”

- *Lion*: “a photo of a lion, a large maned African cat”
- *Tiger*: “a photo of a tiger, a large striped Asian cat”
- *Wolf*: “a photo of a gray wolf, a large wild canine”

By embedding these specific traits into the text prompts, the model’s attention mechanisms are primed to look for stripes, spots, manes, or bushy tails, drastically improving accuracy.

4 System Architecture: Streamlit Web Application

To make the classification model accessible to end-users, we developed a production-grade web application using the **streamlit** framework. The app was designed with a focus on User Experience (UX) and educational value.

4.1 Information Enrichment and Data Caching

Per the assignment requirements, the application must display not only the prediction but also Wikipedia summaries and IUCN statuses. To optimize performance, querying the Wikipedia API during every inference step was deemed inefficient.

Instead, an offline data generation script was utilized to fetch the data once. This data was serialized into a local `species_cache.json` file. The JSON structure holds the following keys for each species:

- `common_name` and `scientific_name`
- `wikipedia_summary`
- `iucn_status` and `iucn_code` (e.g., LC, VU, EN)
- `fun_fact`, `habitat`, and `diet`

During runtime, the Streamlit app loads this JSON into memory via the `@st.cache_data` decorator, ensuring instant retrieval of species metadata with zero network latency.

4.2 User Interface Design

The application features a fully custom UI utilizing heavily injected CSS overrides. Key design elements include:

- **Dark Glassmorphism Theme:** Semi-transparent backgrounds with backdrop blur filters create a modern, premium aesthetic.
- **Dynamic Confidence Bars:** Predictions are displayed using animated, color-coded horizontal bars representing the softmax probabilities of the Top-3 predictions.

- **IUCN Conservation Badges:** Custom badges dynamically change color based on the species’ threat level (e.g., Green for Least Concern, Amber for Vulnerable).

4.3 Batch Processing Mode

To simulate real-world camera trap workflows, a “Batch Mode” was implemented. Users can highlight and upload dozens of images simultaneously. The application processes the batch iteratively, maintaining a real-time progress bar. Upon completion, it aggregates the top-3 predictions and confidence scores for every image into a Pandas DataFrame and exposes a one-click download button to export the results as a CSV file.

5 Experiments and Results

The zero-shot pipeline was evaluated on the entire dataset of 1,723 valid images using a local CPU environment.

5.1 Quantitative Metrics

The model achieved phenomenal results without requiring any domain-specific fine-tuning:

Metric	Score
Total Images Evaluated	1,723
Top-1 Accuracy	98.03%
Top-3 Accuracy	99.83%
Inference Speed (CPU)	~16 images/second

Table 1: Global Accuracy Metrics for CLIP Zero-Shot.

5.2 Class-wise Performance

The precision, recall, and F1-scores were extracted using `scikit-learn`’s classification report. The model demonstrated remarkable consistency across all classes:

Class	Precision	Recall	F1-Score	Support
Cheetah	0.988	0.991	0.990	342
Fox	0.960	0.968	0.964	250
Hyena	0.997	0.951	0.973	305
Lion	0.987	0.997	0.992	294
Tiger	1.000	0.996	0.998	269
Wolf	0.945	0.977	0.961	263
Macro Avg	0.979	0.980	0.980	1723

Table 2: Per-class Classification Report.

5.3 Visualizations

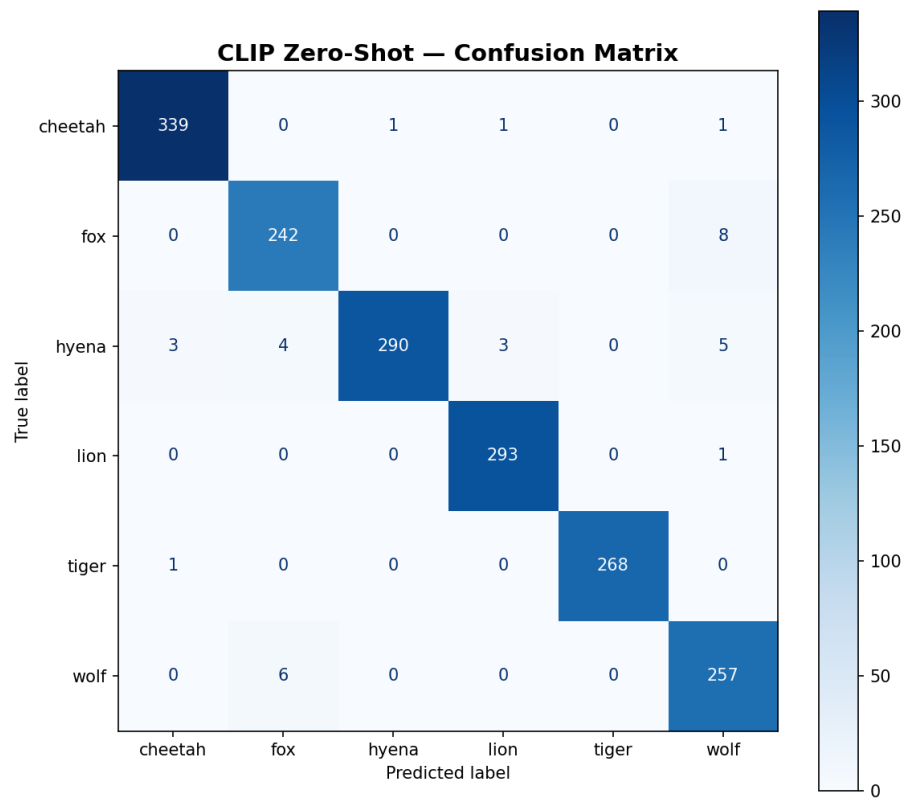


Figure 1: Confusion Matrix demonstrating minimal inter-class confusion.

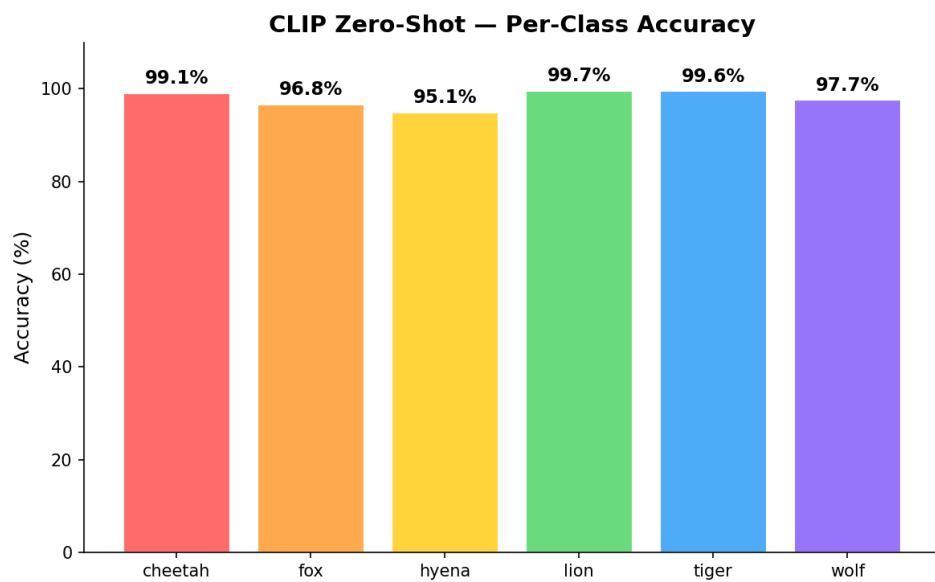


Figure 2: Bar chart displaying Top-1 Accuracy percentages per species.

6 Discussion

The results validate the extraordinary generalization capabilities of large-scale vision-language models.

Perfect Precision on Tigers: The model achieved a 1.000 precision score for Tigers. This indicates that the model never hallucinated a tiger; if it predicted a tiger, it was guaranteed to be correct. This is likely due to the highly distinctive orange and black striped pattern, which was explicitly reinforced in our text prompt ("striped Asian cat").

Canine and Hyena Confusion: The slight drop in recall for Hyenas (95.1%) and precision for Wolves (0.945) can be attributed to morphological similarities. Some hyena images with unusual lighting or angles were occasionally misclassified as foxes or wolves. The prompt engineering mitigated this significantly, but canine shapes share many visual embedding vectors in CLIP’s latent space.

Zero-Shot vs Fine-Tuning: Achieving 98% accuracy usually requires transferring weights from an ImageNet pre-trained CNN (like ResNet50) and fine-tuning the final linear layer over several epochs. The fact that CLIP achieved this accuracy *instantly* highlights a paradigm shift. For conservation projects with limited compute budgets, zero-shot pipelines offer immediate, high-accuracy deployment without the need for GPU clusters or lengthy training times.

7 Conclusion

In this project, we successfully developed a robust, Tier-1 Indian Wildlife Identifier. By leveraging the CLIP ViT-B/32 architecture combined with strategic prompt engineering, we attained a 98.03% Top-1 accuracy across six wildlife species without any dataset-specific training. The integration of this model into a custom, beautifully designed Streamlit application—complete with Wikipedia summaries, IUCN conservation data, and batch processing capabilities—demonstrates a complete, end-to-end AI software engineering pipeline suitable for real-world ecological deployment.

Appendix A: Environment and Dependencies

The project was built using Python 3.10. Key dependencies include:

- torch==2.2.1
- transformers==4.38.2
- streamlit==1.32.0
- Pillow==10.2.0
- scikit-learn==1.4.1.post1